Group Project 3

Determining the Best Combination

DBA5101 Analytics in Managerial Economics | AY2026/2027 Semester 1

Deadline: Sunday, 15 November 2026, 23:59 (Singapore time)

The setting

A website can combine four call-to-action buttons with six media options (three images and three videos). This gives 24 possible combinations, or **arms**. The setting is inspired by the website experiment used in Barack Obama's fundraising campaign.[1]

Your goal is to **maximize the total number of visitors who sign up**. You must learn which arms work well while deciding where to send the next batch of visitors. This is a multi-armed bandit problem: exploration helps you learn, while exploitation uses what you have learned.

Project parameter	Value
Available arms	24 arms, numbered 0 to 23
Visitors per pull	100
Reward per pull	Number of sign-ups among those 100 visitors
Maximum budget per group	1,000,000 pulls = 100,000,000 visitors

Your task

Choose and implement a method for selecting arms. You may use any algorithm or method, but explain why you selected it and how it balances exploration and exploitation. Use the results to explain which arm you would choose and how confidently you can support that choice.

The pull limit is shared by your entire group across all sessions. Test calls also use this budget. Different groups have different arm assignments, so another group's best arm need not be your best arm.

Report and submission

Submit **two separate files on Canvas**, with **one submission per group**. Do not zip the files.

1. **Report in PDF:** at most four pages, excluding appendices. Include all group members' names and student numbers, plus your group username, on the cover page. Explain your method and report the results you obtained. State your pull usage and total sign-ups, and use relevant tables or figures to support your conclusions.
2. **Code in HTML:** use Python or R and export your analysis as an HTML file. Include the code and relevant results needed to understand your analysis. If you use an LLM such as ChatGPT, include the prompts used in your submission.

Do not include your group password in either file.

[1] Background reading: Dan Siroker, Obama's $60 Million Dollar Experiment, Optimizely (29 November 2010).

Using the Cloud Server

Python starter and response reference

Getting started

1. Use https://gp3-bandit-2026.onrender.com with a normal internet connection. NUS Wi-Fi or VPN is not required. Use the experiment username and password assigned to your group; these are separate from your NUS login. Contact the teaching team if you have not received them.
2. Download `server_pull.py` and `GP3 Starter.ipynb` into the same folder. Open the notebook with Python 3.9 or later. The helper uses only the Python standard library. The correct server address is already filled in.
3. Run the offline example first. Keep `RUN_LIVE = False` until you intend to use one pull. Set it to True and enter your assigned credentials to make the example request for arm 2. Adapt the notebook to implement your own arm-selection method.

Minimal Python example

```
from getpass import getpass
from server_pull import configure, status, pull

configure("https://gp3-bandit-2026.onrender.com")
username = input("Group username: ").strip()
password = getpass("Group password: ")
print(status(username, password))  # Uses no pulls
result = pull(username, password, 2)  # One intended pull
print(result)
```

Call `configure` once at the start of a session. After a successful response, a new call consumes another pull. If a response is uncertain, follow the recovery steps on page 3 before starting a new request.

Interpreting a successful response

```
{'Arm': '2', 'NetReward': 455, 'Pull': 30, 'Reward': 22}
```

These values are illustrative, not your group's result.

Field	Meaning
Arm	The arm selected for this request.
Reward	Sign-ups among this batch of 100 visitors. A reward of 22 means a 22% batch conversion rate.
NetReward	Total sign-ups accumulated by the group account, including this batch.
Pull	Zero-based completed-pull index. Pull = 0 is the first pull; Pull = 30 means 31 pulls have completed.

`status(username, password)` reports `PullsUsed`, `NetReward`, `MaxPulls` and `Open` without consuming a pull. A closed experiment accepts no new pulls.

Running Experiments Safely

Shared accounts, recovery and other programming languages

Coordinate your group

Use one sequential experiment driver for the group. Allow at least **0.25 seconds between calls**. The server permits at most five requests per second per group, shared by all members. The full one-million-pull budget remains shared across all sessions, including test calls.

Save each response as you work. Restarting a notebook does not reset the account or restore its budget. Keep passwords out of saved code, outputs, URLs and submitted files. The server closes to new pulls at the project deadline; complete experiments in time to prepare your report.

Recover an uncertain request

The helper never retries automatically. A timeout, connection failure or temporary server error may occur after a pull has been recorded. Do not choose a new arm or advance the counter until that request is resolved.

1. Keep the arm, request_id and expected_pull reported by UncertainPullError. Wait at least one second if requests were rate-limited. Do not run configure again while a request is pending.
2. In the same Python session, repeat the pull for the same arm and account. The helper retains the pending request. If you restarted the kernel, configure the same server and supply the saved recovery fields explicitly:

```
result = pull(username, password, saved_arm,
              request_id=saved_request_id,
              expected_pull=saved_expected_pull)
```

3. An identical retry returns the original result if the request completed; otherwise it makes that one intended pull. If another member used the same counter, coordinate, refresh status and then decide on a new request. Contact the teaching team if the state remains unclear.

Using R or another HTTP client

Use the same HTTPS base address and group account. Send credentials in headers: `X-Group: your_group_username` and `Authorization: Bearer your_group_password`. Never put them in the URL.

Request	Purpose and data
GET /v1/status	Read PullsUsed, NetReward, MaxPulls and Open; no pull is consumed.
POST /v1/pull	Send Content-Type: application/json with arm (0-23), expected_pull (current PullsUsed) and request_id (a new UUID). The successful response contains the result object shown on page 2.

For an uncertain request, reuse exactly the same UUID, counter and arm. Use a new UUID only for a new intended pull. Apply the same pacing and recovery rules as the Python helper.

Help

Kerun Xu: e1521287@u.nus.edu
Abilasha Swaminathan: abilasha@u.nus.edu